GeoIPS Workshop 2026: Sectors

Andrew Thorpe

20260721

Overview

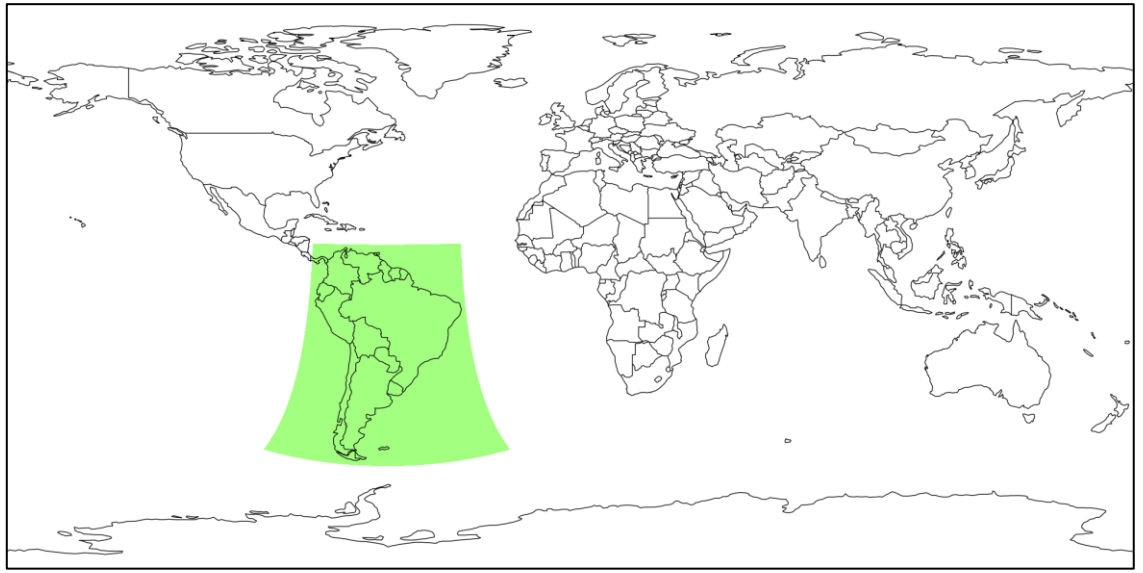
- Geographic sectors in GeoIPS
- Capabilities of sectors in GeoIPS:
  - Static Sectors
  - Dynamic Sectors
- Sectors and Sectoring backend
- Sector Examples:
  - Static: Vera Cruz Oil spill
  - Dynamic: Iceberg A23

GeoIPS sectors

- YAML defined sectors
- Set style helps interchange projection type, size, and resolution
- Wide coverage of applications between static and dynamic sectors
- Interchangeable; same sector for different products, readers, and data sources
- 55 static sectors and 23 dynamic sectors
- Defined structure and format, see documentation: Extend GeoIPS with a Static Sector

Tools for sectors

- Tools to visualize sectors:
  - geoips test sector <sector_name>
    - Displays sector in defined projection
  - geoips test sector <sector_name> --overlay:
    - Overlays sector onto global equidistant cylindrical projection



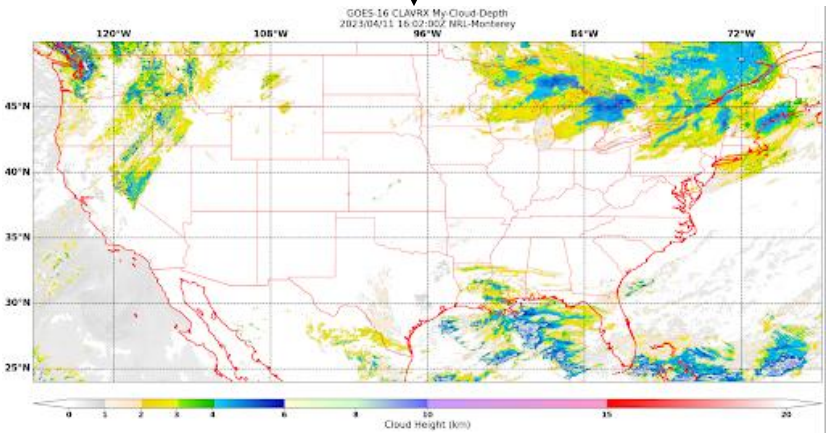
Static Example

What if I want to capture a static region on Earth, and use it for any sensor, product, or algorithm?

Static Sector Capabilities

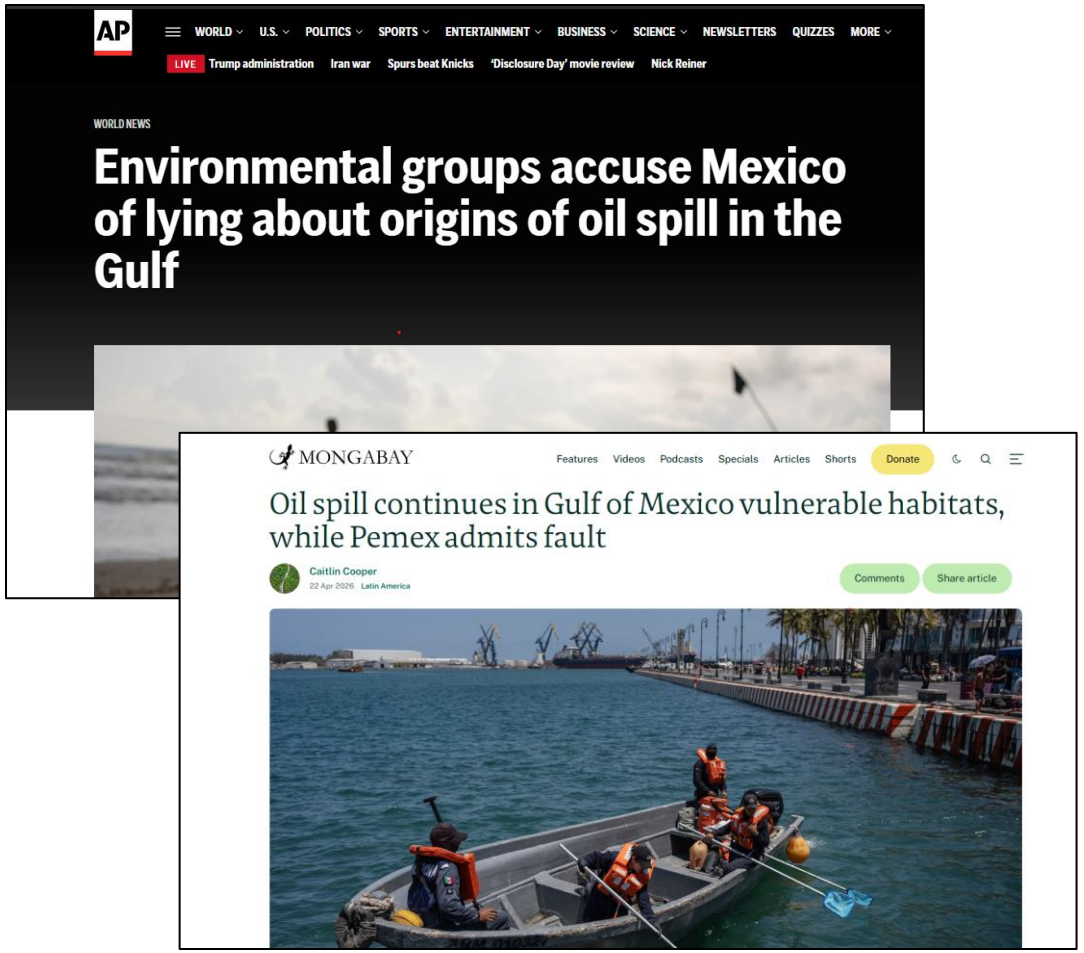
- Fixed in space and time
- Tunable parameters:
  - Center latitude & longitude
  - Projection type (EQC)
  - Units (meters), used in resolution
  - Resolution of each pixel (in user defined units; km/m)
  - Shape (height/width in pixels)
  - Center (almost always 0,0)
- Tunable metadata:
  - Region description; can be used in output metadata and formatting
  - Continent (EX: NorthAmerica)
  - Country (EX: USA)
  - Area (EX: WestCoast)
  - Subarea (EX: DeathValley)
  - State (EX: California)
  - City (EX: Shoshone)
- Downsides:
  - Custom projections are hard to implement

```yaml
interface: sectors
family: area_definition_static
name: my_conus_sector
docstring: "My CONUS Sector"
metadata:
  region:
    continent: NorthAmerica
    country: UnitedStates
    area: x
    subarea: x
    state: x
    city: x
spec:
  area_id: my_conus_sector
  description: CONUS
  projection:
    a: 6371228.0  # The average radius of the Earth in Meters
    lat_0: 37.0  # The center latitude point
    lon_0: -96.0  # The center longitude point
    proj: eqc # Describes the Projection Type (from PROJ Projections)
    units: m
  resolution:
    - 3000 # The resolution of each pixel in meters (x, y)
    - 3000
  shape:
    height: 1000  # The height of your sector in pixels
    width: 2200  # The width of your sector in pixels
  center: [0, 0]  # The center x/y point of your sector. Almost always [0, 0]
```



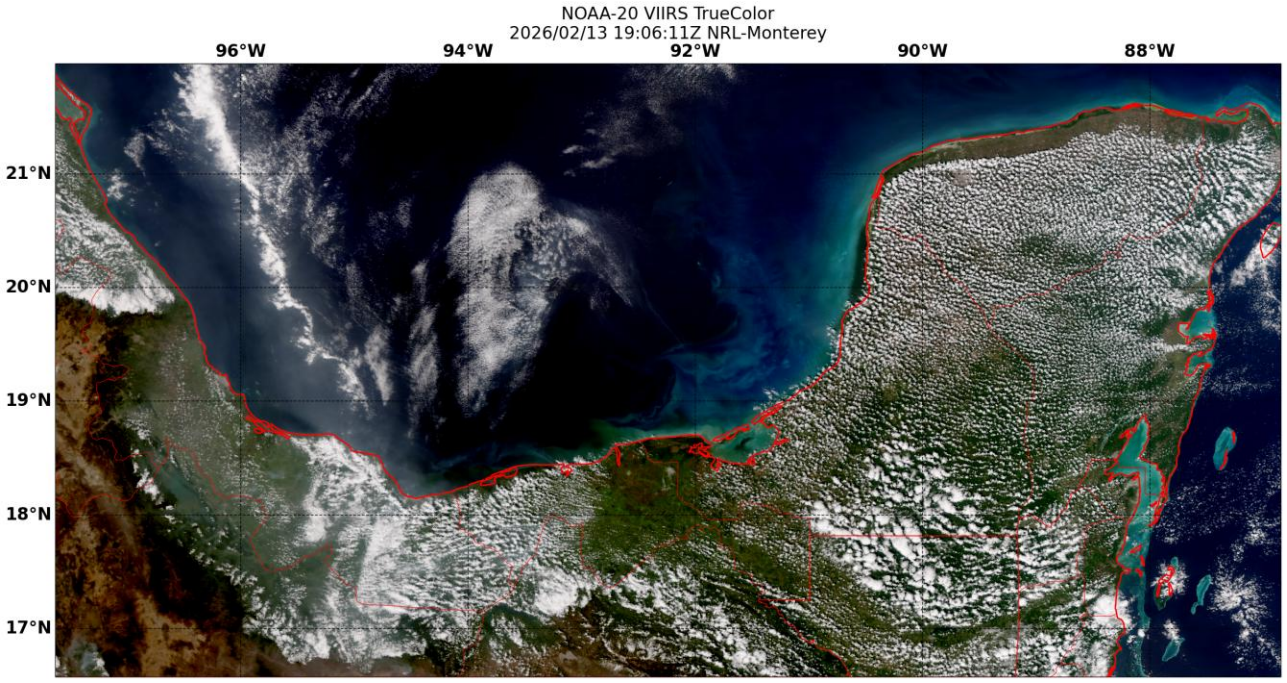
Problem: Oil Spill outside of Veracruz

- New coverage of an oil spill in February, March of 2026
- Detections of oil spills from beaches, fishing communities, and surrounding habitats
- Government owned Pemex initially denied leakage in early cases, but admitted fault in April



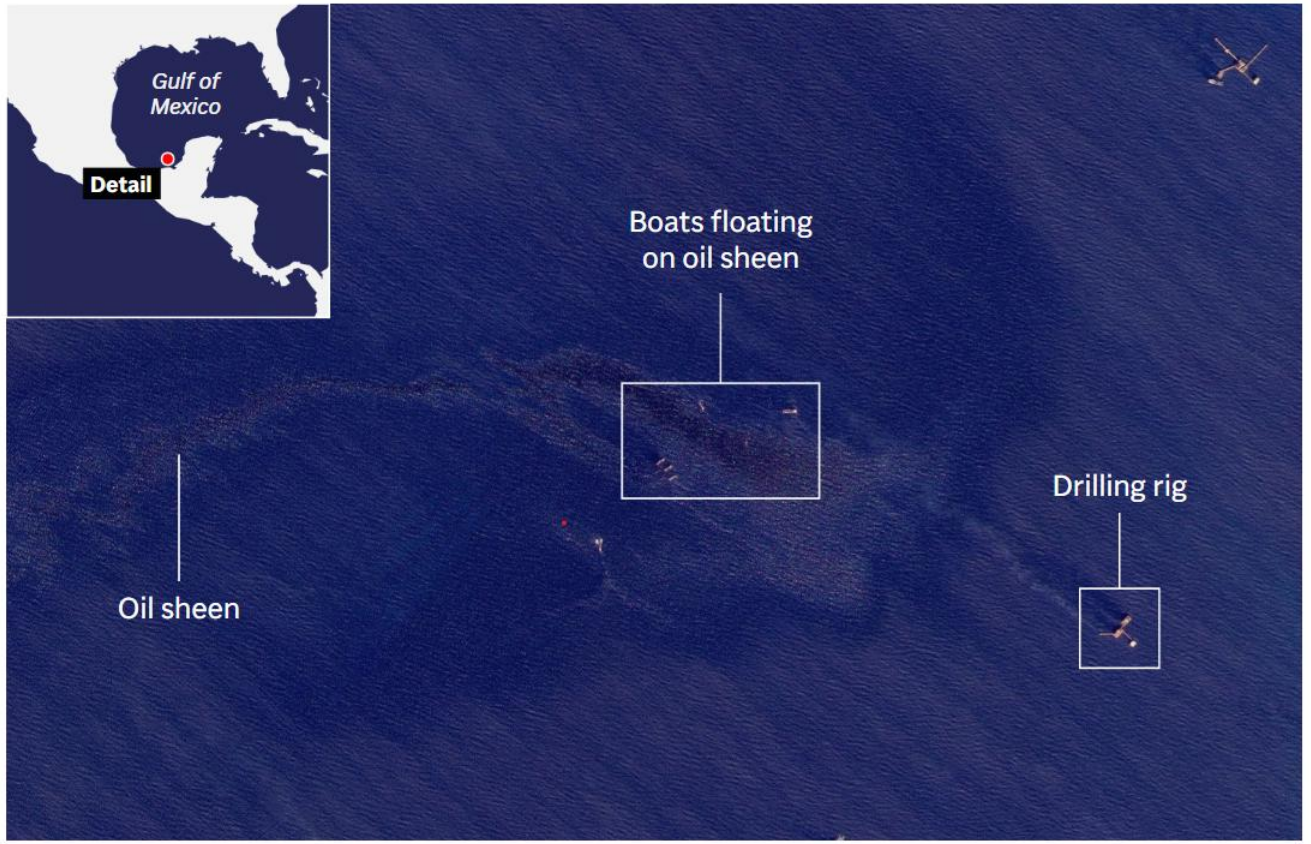
Static Sector Example

- First pass failed using VIIRS data to identify a sector or detection
- Resolution limited…!
- Reached out to AP New’s Will Jarrett, success, but used proprietary data source
- Sector derived!



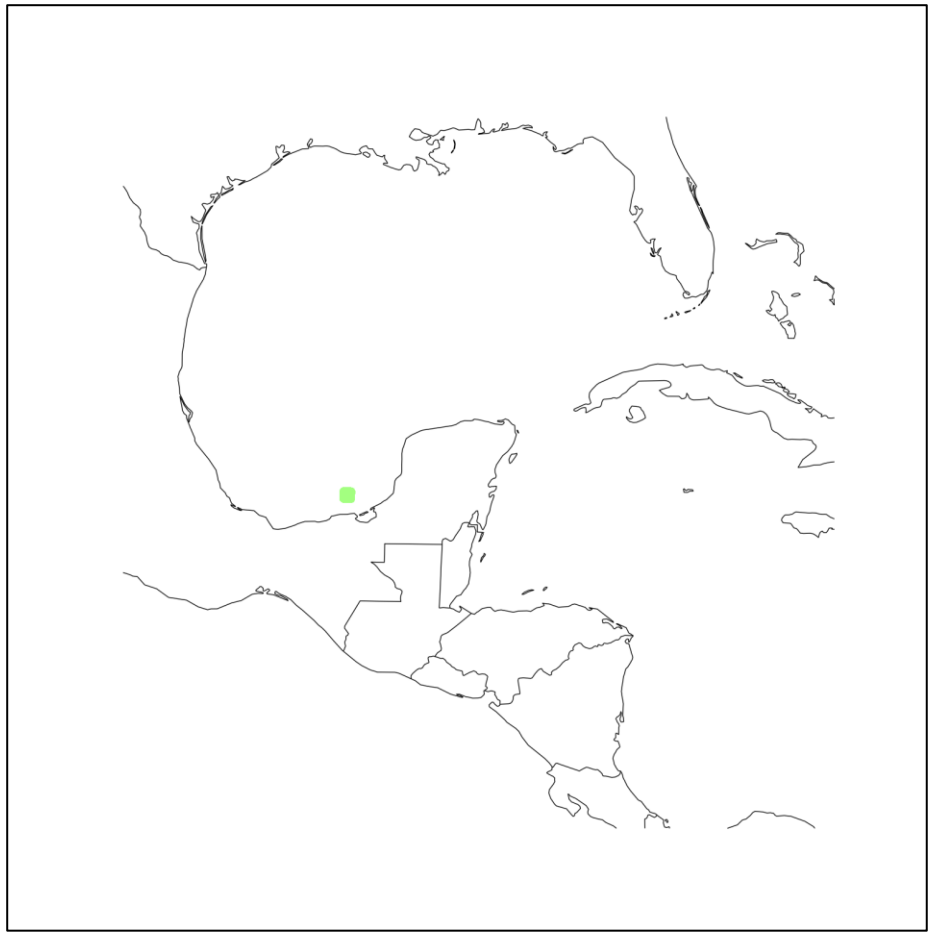
Problem: Oil Spill outside of Veracruz

- News sources, and scientists cited using satellite imagery for oil spill identifications
- Only AP News displayed imagery with minimal information…..
- Goal: Create a static sector to capture oil spills from Pemex oil rigs…?



Static Sector Example: Derived Values

- Time: 20260213T171453Z
- Latitude: 19.27517
- Longitude: -92.22473
- Resolution: 50 meter
- Projection: eqc
- Size: 20 km goal diameter, 400 by 400 size

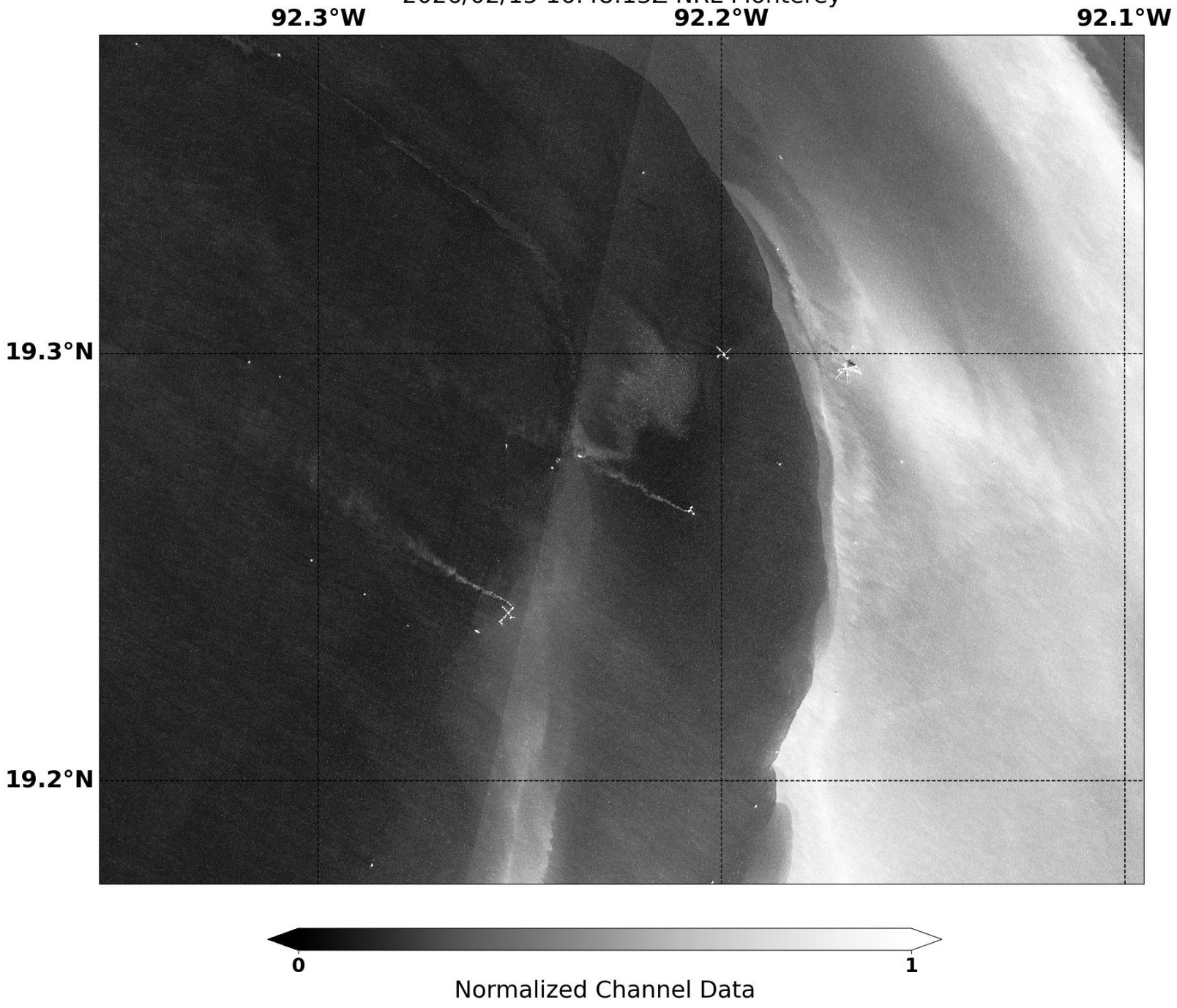


Static Sector: Applied

- Derived sector can be applied to any product!

```
run_procflow $@ \
    --procflow single_source \
    --log info \
    --reader_name sent2_llc_j2 \
    --product_name B03 \
    --filename_formatter geoips_fname \
    --output_formatter  imagery_annotated \
    --minimum_coverage 0 \
    --sector_list vcruz
#
```

```
interface: sectors
family: area_definition_static
name: vcruz
docstring: "Vera Cruz Bay"
metadata:
  region:
    continent: NorthAmerica
    country: Mexico
    area: GulfofMexico
    subarea: VeraCruz
    state: x
    city: x
spec:
  area_id: vcruz
  description: Vera Cruz Bay
  projection:
    a: 6371228.0
    lat_0: 19.27517
    lon_0: -92.22473
    proj: stere
    units: m
  resolution:
    - 25
    - 25
  shape:
    height: 1600
    width: 1600
  center: [0, 0]
```

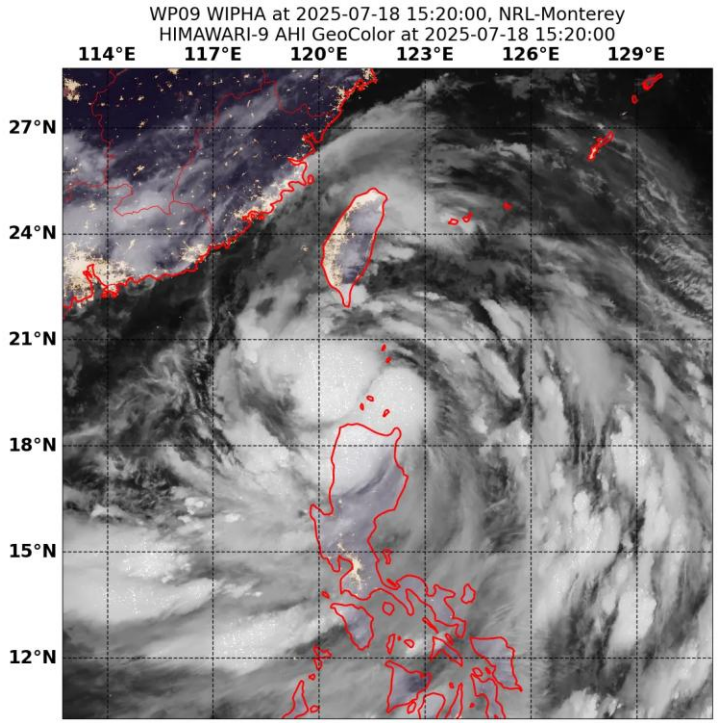

SENTINEL-2 MSI B03
2026/02/15 16:48:13Z NRL-Monterey
92.3°W
92.2°W
92.1°W
19.3°N
19.2°N
0
1
Normalized Channel Data

Dynamic Example

What if I want to capture a *dynamic* region or *transient* phenomenon on Earth, and use it for any sensor, product, or algorithm?

Dynamic Sector Capabilities

- Capture transient activity, can be dynamic in space and/or time
- Work in conjunction with track files and parsers
- Tunable parameters:
  - Pixel width (meters)
  - Pixel height (meters)
  - Number lines (pixels)
  - Number samples (pixels)
- Tunable metadata:
  - Docstring
- Downsides:
  - Not well documented



GeoIPS utilizes dynamic sectors for Tropical Cyclones
(imagery via TCWeb)

Dynamic Sector Example

- Iceberg tracking: A23A, long-living iceberg
- Only visible by polar satellites, and transient in time and location
- Application: Tracking phytoplankton blooms due to iceberg mineral (iron) deposits

Changes needed for implementation

- Created an iceberg dynamic sector file (yaml)
- Created a sample iceberg track file (csv)
- Created an iceberg dynamic track file parser (csv to list)
- Modified sector utilities to create a dynamic AreaDefinition list for icebergs from a parsed track files (list to AreaDefinition)
- Modified sector metadata generators class to accept icebergs as an additional parser class

Dynamic Sector Components: TrackFile

- Created a multi-time file from daily iceberg reports from the USNIC
- Formatted file that contains:
  - Latitude (required)
  - Longitude (required)
  - Time (required)
  - Relevant metadata

	A	B	C	D	E	F	G	H	I	J
1	Iceberg	Length (NM)	Width (NM)	Latitude	Longitude	Area (sqMI)	Area (sqNM)	Area (sqKM)	Last Update	
2	A23A	26	22	-52.42	-40.24	518.63	391.63	1343.24	12/5/2025	
3	A23A	26	22	-52.74	-39.89	456.4	344.64	1182.07	12/12/2025	
4	A23A	26	22	-52.93	-40.1	456.4	344.64	1182.07	12/18/2025	
5	A23A	26	22	-53.04	-40.27	456.4	344.64	1182.07	12/26/2025	
6	A23A	26	22	-53.07	-40.37	456.4	344.64	1182.07	1/2/2026	
7	A23A	26	22	-52.98	-40.88	399.64	301.77	1035.06	1/9/2026	
8	A23A	26	22	-52.74	-41.36	205.35	155.07	531.86	1/16/2026	
9	A23A	26	22	-52.31	-41.49	205.35	155.07	531.86	1/23/2026	
10	A23A	26	22	-51.64	-41.41	216.57	163.54	560.91	1/30/2026	
11										
12										

Dynamic Sector Components: Trackfile parser

- Translates data from trackfile into a list of dictionaries
- Minimum required information:
  - Latitude
  - Longitude
  - Time
  - Name

```
interface = "sector_metadata_generators"
family = "iceberg"
name = "iceberg_parser"

def call(trackfile_name):
    """Parse a sample iceberg csv file.

    Parameters
    ----------
    trackfile_name: str, or path
        * Path to trackfile csv file with the format:
    """
    csv_parser = pd.read_csv(trackfile_name)

    all_fields = []

    for i in csv_parser.iterrows():
        tmp_fields = i[1]
        fields = {}

        fields["clat"] = tmp_fields["Latitude"]
        fields["clon"] = tmp_fields["Longitude"]
        fields["time"] = pd.to_datetime(tmp_fields["Last Update"]).to_pydatetime()
        all_fields.append(fields)

    iceberg_name = tmp_fields["Iceberg"]
    if "iceberg" not in iceberg_name.lower():
        iceberg_name = iceberg_name.upper() + "_ICEBERG"
    return all_fields, iceberg_name, fields["time"].strftime("%Y"), "BEST"
```

Dynamic Sector Components: Parser to AreaDefinition

- Iteratively transforms the iceberg parsed information into area definitions
- Set start and end datetimes, which help sector the dataset in the end
- Calls the center_coordinate sector spec generator

```
    elif "iceberg" in final_storm_name.lower():
        LOG.info("STARTING setting ICEBERG area_defs")
        # raise
        for fields in all_fields:

            area_defs += [set_iceberg_area_def(fields, final_storm_name, tc_spec_template)]
        return area_defs, allowed_aid_types
```

```
def set_iceberg_area_def(fields, storm_name, sector_spec_template):
    """Set iceberg area definitions from fields."""
    clat = fields["clat"]
    clon = fields["clon"]
    # TBD: reformat these to be part of a parser?
    area_id = storm_name
    long_description = "iceberg {}".format(storm_name.lower())

    tc_template_plugin = sectors.get_plugin(sector_spec_template)
    # template_func_name = tc_template_plugin["spec"]["sector_spec_generator"]["name"]
    template_args = tc_template_plugin["spec"]["sector_spec_generator"]["arguments"]

    # Only one sector spec generator to choose from: 'center_coordinates'
    template_func = sector_spec_generators.get_plugin("center_coordinates")
    template_args["area_id"] = area_id
    template_args["long_description"] = long_description
    template_args["clat"] = clat
    template_args["clon"] = clon
    area_def = template_func(**template_args)

    area_def.sector_start_datetime = fields["time"]
    area_def.sector_end_datetime = fields["time"]
    area_def.sector_type = "iceberg"
    area_def.sector_info = {}

    # area_def.description is Python3 compatible,
    # and area_def.name is Python2 compatible
    area_def.description = long_description

    area_def.sector_info["storm_name"] = storm_name
    area_def.sector_info["final_storm_name"] = storm_name
    LOG.info("Finished iceberg area def")
    return area_def
```

Dynamic Sector Imagery

```
geoips run single_source \
    $@ \
    --reader_name viirs_sdr_hdf5 \
    --product_name TrueColor \
    --filename_formatter geoips_fname \
    --output_formatter imagery_annotated \
    --trackfile_parser iceberg_parser \
    --log debug \
    --trackfiles AntarcticIcebergs_202512_202601.csv \
    --tc_spec_template iceberg_dyn
```